

Automated Discovery of Quantum Error-Correcting Codes

State-of-the-Art Review

AutoQEC review draft

Generated 2026-06-08 from the AutoQEC knowledge base (51-reference survey).

Scope

This report assesses **automated / algorithmic discovery of quantum error-correcting (QEC) codes**: methods that search, optimize, or learn good codes rather than constructing them by hand. The review is organized *by technical approach* the state of the art and trade-offs are given inside each approach. It is aimed at internal technical decision-making for the AutoQEC search project, not as a general QEC tutorial. A companion report covers the lifted-product / quantum-Tanner family as a concrete search target.

1. What and Why

A quantum error-correcting code encodes logical qubits into more physical qubits so that errors can be detected and corrected. The central practical problem is that *finding* a good code — one with high distance, high rate, sparse low-weight checks, a tractable decoder, and a useful logical-gate structure all at once — is a vast combinatorial search. Hand construction has produced the canonical families (surface, color, bivariate-bicycle, lifted-product), but the space of possibilities is far larger than human enumeration can cover, and the best code for a given noise model or hardware layout is rarely the textbook one [1], [2].

Automated code discovery reframes this as an optimization problem and matters now for three reasons. First, near-term fault tolerance needs strong *finite-length* instances tuned to real noise and connectivity, not just good asymptotic families. Second, candidate generation has become cheap — learned controllers can propose thousands of codes — which shifts the binding constraint onto *validation*: exact distance and equivalence checking now dominate the cost [3], [4], [5]. Third, the same machinery now reaches code classes that resist hand construction, including non-CSS, approximate, and dissipative codes [6], [7], [8].

What makes this different from hand construction is the locus of effort. Manual work proves a family has good asymptotic parameters; automated discovery instead invests in *shrinking and shaping the search space* so a generic optimizer finds strong finite instances, and in *evaluating candidates fast enough* to keep the loop running. Across the literature the recurring workflow is the same five stages below; the approaches in §3 differ mainly in how they implement stage 3.

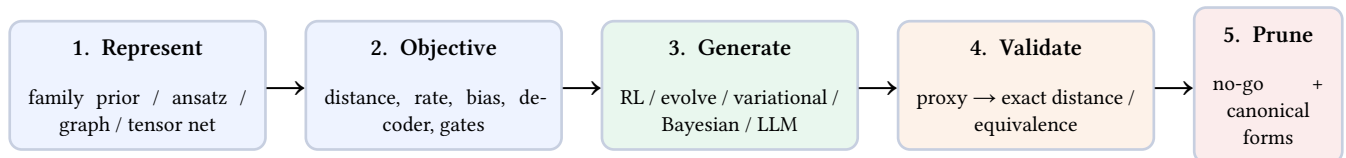


Figure 1: The automated QEC code-discovery loop. The approaches in §3 differ chiefly in stage 3; stage 4 is the shared throughput bottleneck and stage 5 feeds structure theorems back as constraints.

2. Technical Approaches

The field has moved through three eras: **representation engineering** (1997–2011), where the win came from searching in a better coordinate system; **noise-aware design** (2017–), where realistic channels became routine search inputs; and the current era of **learned controllers and algebraic family mining** (2022–2026), where RL, surrogate models, and even LLM-guided program evolution wrap around classical exact validators. The six approaches below are roughly ordered by that lineage, followed by the cross-cutting validation infrastructure they all depend on.

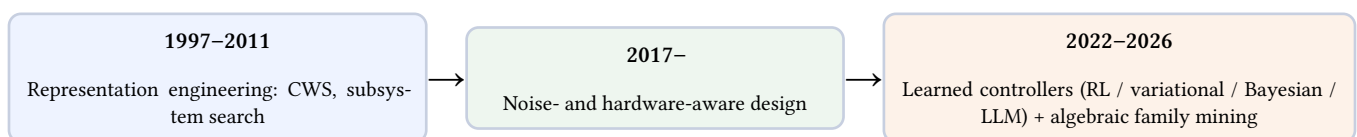


Figure 2: Field landscape: the searched object widened from “the code” to “code + encoder + circuit + layout,” and the generator became a learned controller around exact validators.

2.1. Representation-constrained combinatorial search

What it is. Recast discovery as a structured search in a better coordinate system rather than over raw stabilizer tableaux: codeword-stabilized (CWS) graph states with induced classical error patterns and clique-style reductions, or subsystem codes parameterized by physically allowed measurement operators.

State of the art. The CWS framework turned code discovery into graph/clique search and produced both new codes and non-existence results such as the absence of a $((7,3,3))$ CWS code [9], [10]; the first nonadditive code beating the best additive code established that the stabilizer ansatz is too narrow [11]; subsystem-code search optimizes over measurement operators directly [12].

Strengths

- Exposes codes additive/stabilizer search misses [11].
- Same machinery yields existence *and* impossibility results [10].

Limitations

- Strongly representation-dependent; needs symmetry reduction first [10].
- Combinatorial blow-up confines it to small n .

2.2. Reinforcement learning

What it is. An agent incrementally composes or edits a code — often jointly with its encoder, gate set, and connectivity — under a reward such as the Knill–Laflamme conditions or logical error rate, with the noise and hardware model in the loop.

State of the art. Joint code+encoder discovery scales to 20 qubits / distance 5 via a vectorized Clifford simulator [13]; the Quantum-Lego line saturates the CSS linear-programming bound on 13 qubits and finds best-known biased-noise codes near 20 qubits, now with a device-in-the-loop hybrid [14], [15]; tensor-network RL beats random search by $65\times$ [16]; RL also performs stabilizer weight reduction (1–2 orders of magnitude overhead saving) [17], fault-tolerant state-prep circuits [18], and bosonic autonomous codes [19].

Strengths

- Searches code + encoder + circuit jointly [13].
- Folds noise/hardware constraints directly into the reward [14], [15].
- Strong empirical results to 20 qubits.

Limitations

- Sensitive to reward design and simulator throughput [13].
- Train/deploy noise mismatch; sample-expensive [20].
- Optimality is hard to certify.

2.3. Evolutionary and genetic search

What it is. Encode codes or stabilizer circuits as genomes (binary strings, circuit programs), then mutate, recombine, and select on a distance- or performance-based fitness.

State of the art. A binary-string genome matches codetables.de distance for $n \leq 20$, gains on biased noise, and contributes the QDistEvol distance heuristic [20]; circuit evolution rediscovers codes equivalent to the 5-qubit perfect code, Shor’s code, and the 7-qubit color code [21].

Strengths

- Simple and competitive with RL at small–medium n [20].
- Naturally couples generation to a distance evaluator [20].

Limitations

- Fitness = repeated distance evaluation, the dominant cost [20].
- Representation and scaling limits keep it to small codes.

2.4. Variational, gradient, and surrogate optimization

What it is. Make code synthesis differentiable or surrogate-driven: turn the Knill–Laflamme conditions into a variational loss, parameterize codewords on a manifold and follow gradients, or train a model that predicts logical error rate without simulation.

State of the art. VarQEC (re)discovers symmetric/asymmetric codes and new non-stabilizer $((6,2,3)) / ((7,2,3))$ codes [2]; distinguishability-loss training learns noise-tailored encoders demonstrated on IBM and IQM hardware [22]; graphical VGQEC interpolates between code families [23]; learned concatenation [24] and ML-inspired approximate amplitude-damping codes [7] extend the frontier; Stiefel-manifold Riemannian optimization [25] and Wirtinger-gradient codeword optimization [26] cover the gradient route; a Bayesian-optimization loop with a chain-complex neural surrogate discovers competitive $[[144,36]]$ and $[[144,16]]$ bivariate-bicycle codes without simulation [27].

Strengths

- Surrogates / differentiability attack the per-candidate evaluation cost [25], [27].
- Reaches approximate and noise-adaptive codes [2], [7].
- Demonstrated on real hardware [22].

Limitations

- Surrogate predictions still need exact confirmation [27].
- Approximate codes lack clean $[[n,k,d]]$ certification [7].
- Gradient methods are sensitive to initialization and penalties [26].

2.5. Algebraic family mining

What it is. Fix a structured family — bivariate / multivariate bicycle, two-block group-algebra, lifted-product — and search its parameters with strong algebraic priors and exact validation; the generator is increasingly an LLM or program-evolution loop.

State of the art. Coprime bivariate-bicycle codes with explicit cold-atom layouts [28]; LLM-guided evolutionary search over BB-generating programs with a certify-and-deduplicate pipeline [29]; BB sequences from covering graphs [30]; multivariate [31] and trivariate [32], [33] generalizations; a multivariate-multicycle unification with complete single-shot decoding [34]; the two-block group-algebra formalization [35] and non-CSS “mirror” codes [6]; short transversal-friendly codes [36]; and finite-length distance results for lifted-product / lifted-Tanner families [37], [38], [39].

Strengths

- Exact-verifiable, hardware-mappable, moderate-length (n in the 100s) instances [28], [34].
- Rich algebraic structure keeps both search and decoding tractable [34].

Limitations

- Some families are provably asymptotically bad [37].
- Risk of overfitting to algebraically pleasant dead ends.
- Logical-gate structure vs. finite-length distance is often a trade [37].

2.6. Autonomous and open-system discovery

What it is. Optimize the dissipative code-plus-control object — code subspace, induced decay rates, and control Hamiltonian — directly against a physical Lindbladian, rather than a static code subspace.

State of the art. Adjoint-optimization search discovers autonomous bosonic codes such as the $\sqrt{3}$ code with a hardware-efficient superconducting implementation [8]; a full open-system search over code space, decay rates, and control finds codes that beat the binomial code on perturbed systems [40].

Strengths

- Targets the real dissipative object, not just the static code [8].
- Hardware-efficient bosonic results [8].

Limitations

- Physics-model-specific; expensive open-system simulation [40].
- Immature decoder / fault-tolerance story.
- Narrow scope (bosonic / autonomous) so far.

2.7. Cross-cutting: validation, equivalence, and pruning

What it is. Not a discovery method but the shared backend every approach above depends on: exact and heuristic distance computation, equivalence canonicalization for deduplication, theorem-based pruning, and community libraries / databases.

State of the art. Distance tooling spans QDistRnd [41], fast exact Brouwer–Zimmermann [3], the codeDistance benchmark of exact and heuristic methods [4], Monte-Carlo upper bounds [42], and QUBO/annealing formulations [43]; ZX-calculus canonical forms enable deduplication [5]; classification and no-go theorems prune whole regions [10], [37], [44]; and the qLDPC library [45], SSIP [46], quantumcodes.info [47], and the Error Correction Zoo [48] provide shared infrastructure, with a comprehensive AI-for-QEC review [49] and SMT/Coq verification [50] bracketing the field.

Strengths

- Turns “looks good” into “is good”; canonical forms deduplicate [3], [5].
- Theorems prune whole regions before any search runs [44].

Limitations

- Exact distance / equivalence is the throughput bottleneck once generation is cheap [4].
- Few no-go theorems are expressed as machine-checkable constraints [44].

2.8. At-a-glance comparison

Approach	Scalability	Verifiability / cost	Maturity	Best-fit use case
Representation / combinatorial	Small n	Exact, but blows up [10]	Mature concept	Small exceptional codes; impossibility results

Approach	Scalability	Verifiability / cost	Maturity	Best-fit use case
Reinforcement learning	20 qubits [13]	Empirical; costly to certify	Active, early	Code + encoder + circuit, noise-tailored
Evolutionary / genetic	Small-medium n [20]	Fitness = distance eval	Active, early	Small biased-noise codes
Variational / gradient / surrogate	Surrogate scales [27]	Needs exact confirmation	Active, early	Approximate / noise-adaptive; qLDPC surrogates
Algebraic family mining	Moderate n (100s) [28]	Exact-verifiable [34]	Maturing	Hardware-mappable LDPC memories
Autonomous / open-system	Few modes [40]	Physics-sim costly	Early	Bosonic / dissipative hardware codes

3. Open Problems

No.	Problem	Why it matters	Who could solve it	Urgency
1	Scale exact validation	Distance, equivalence, and logical-gate checking dominate cost once candidate generation is cheap, capping every search loop's throughput	Distance-algorithm + equivalence groups (Hernando, Webster, Khesin)	Critical
2	Unified multi-objective scoring	No single objective jointly scores code parameters, encoder complexity, decoder compatibility, and logical-gate structure, so methods optimize one axis and regress on others	Method groups building the search loops (RL / variational / Bayesian)	Critical
3	Shared finite-length benchmarks	Without common tasks, RL, variational, Bayesian, and combinatorial methods cannot be compared, so progress claims are not commensurable	Community / benchmark maintainers (codeDistance, code zoo)	High
4	Family priors: broad yet tractable	Priors must admit surprises while preserving algebraic structure that keeps validation feasible; too narrow misses codes, too broad stalls	Algebraic-family + LLM-search groups (IBM, Chengyu)	High
5	Operationalize no-go theorems	Impossibility and classification results could prune the search space but are rarely expressed as explicit machine-checkable constraints	Theory groups (Haah; Postema; CWS line)	Medium
6	Extend cleanly beyond CSS stabilizers	Non-CSS, approximate / noise-adaptive, and dissipative classes need decoders and FT gadgets to match their code-discovery progress	Khesin; Liu; Ashhab / autonomous-QEC groups	Medium

Bottom line for AutoQEC

The field's center of gravity has shifted from “find a new code somewhere” to “find the best code inside a constrained family that matches a noise model, hardware model, and verification budget.” The most successful workflows combine a structured representation or family prior, a fast proxy plus an exact validator, and a theorem-backed pruning rule. For a search project the highest-leverage investment is not a cleverer generator but a faster, broader **validator** — exact distance, equivalence, and logical-gate detection — since that is now the throughput bottleneck. Distilled in one sentence: *progress usually comes from reducing the search space more intelligently than the objective space* [3], [4], [9], [37].

References

- [1] A. Robertson, C. Granade, S. D. Bartlett, and S. T. Flammia, “Tailored Codes for Small Quantum Memories,” *Physical Review Applied*, vol. 8, no. 6, p. 64004, 2017, doi: [10.1103/PhysRevApplied.8.064004](https://doi.org/10.1103/PhysRevApplied.8.064004).
- [2] C. Cao, C. Zhang, Z. Wu, M. Grassl, and B. Zeng, “Quantum Variational Learning for Quantum Error-Correcting Codes,” *Quantum*, vol. 6, p. 828, 2022, doi: [10.22331/q-2022-10-06-828](https://doi.org/10.22331/q-2022-10-06-828).
- [3] F. Hernando, G. Quintana-Ortí, and M. Grassl, “Fast Algorithms and Implementations for Computing the Minimum Distance of Quantum Codes,” *ACM Transactions on Quantum Computing*, vol. 7, pp. 1–19, 2026, doi: [10.1145/3795877](https://doi.org/10.1145/3795877).
- [4] M. Webster, A. Jacob, and O. Higgott, “Distance-Finding Algorithms for Quantum Codes and Circuits.” [Online]. Available: <https://arxiv.org/abs/2603.22532>
- [5] A. B. Khesin and A. Li, “Equivalence Classes of Quantum Error-Correcting Codes.” [Online]. Available: <https://arxiv.org/abs/2406.12083>
- [6] A. B. Khesin and J. Z. Lu, “Mirror codes: High-threshold quantum LDPC codes beyond the CSS regime.” [Online]. Available: <https://arxiv.org/abs/2603.05496>
- [7] S. Liu, S. Zhou, Z.-W. Liu, and J. Yi, “Noise-strength-adapted approximate quantum codes inspired by machine learning.” [Online]. Available: <https://arxiv.org/abs/2503.11783>
- [8] Z. Wang, T. Rajabzadeh, N. Lee, and A. H. Safavi-Naeini, “Automated discovery of autonomous quantum error correction schemes,” *PRX Quantum*, vol. 3, p. 20302, 2022, doi: [10.1103/prxquantum.3.020302](https://doi.org/10.1103/prxquantum.3.020302).
- [9] A. Cross, G. Smith, J. A. Smolin, and B. Zeng, “Codeword Stabilized Quantum Codes,” *IEEE Transactions on Information Theory*, vol. 55, no. 1, pp. 433–438, 2009, doi: [10.1109/TIT.2008.2008136](https://doi.org/10.1109/TIT.2008.2008136).
- [10] I. L. Chuang, A. W. Cross, G. Smith, J. A. Smolin, and B. Zeng, “Codeword Stabilized Quantum Codes: Algorithm and Structure,” *Journal of Mathematical Physics*, vol. 50, no. 4, p. 42109, 2009, doi: [10.1063/1.3086833](https://doi.org/10.1063/1.3086833).
- [11] E. M. Rains, R. H. Hardin, P. W. Shor, and N. J. A. Sloane, “A Nonadditive Quantum Code,” *Physical Review Letters*, vol. 79, no. 5, pp. 953–954, 1997, doi: [10.1103/PhysRevLett.79.953](https://doi.org/10.1103/PhysRevLett.79.953).
- [12] G. M. Crosswhite and D. Bacon, “Automated Searching for Quantum Subsystem Codes,” *Physical Review A*, vol. 83, no. 2, p. 22307, 2011, doi: [10.1103/PhysRevA.83.022307](https://doi.org/10.1103/PhysRevA.83.022307).
- [13] J. Olle, R. Zen, M. Puviani, and F. Marquardt, “Simultaneous Discovery of Quantum Error Correction Codes and Encoders with a Noise-Aware Reinforcement Learning Agent,” *npj Quantum Information*, vol. 10, no. 1, 2024, doi: [10.1038/s41534-024-00920-y](https://doi.org/10.1038/s41534-024-00920-y).
- [14] V. P. Su, C. Cao, H.-Y. Hu, Y. Yanay, C. Tahan, and B. Swingle, “Discovery of Optimal Quantum Error Correcting Codes via Reinforcement Learning,” *Physical Review Applied*, vol. 23, p. 34048, 2025, doi: [10.1103/PhysRevApplied.23.034048](https://doi.org/10.1103/PhysRevApplied.23.034048).
- [15] Y. Yanay, “Learning Better Error Correction Codes with Hybrid Quantum-Assisted Machine Learning.” [Online]. Available: <https://arxiv.org/abs/2601.08014>
- [16] C. Mauron, T. Farrelly, and T. M. Stace, “Optimization of Tensor Network Codes with Reinforcement Learning,” *New Journal of Physics*, vol. 26, p. 23024, 2024, doi: [10.1088/1367-2630/ad23a6](https://doi.org/10.1088/1367-2630/ad23a6).
- [17] A. Y. He and Z.-W. Liu, “Discovering highly efficient low-weight quantum error-correcting codes with reinforcement learning.” [Online]. Available: <https://arxiv.org/abs/2502.14372>
- [18] R. Zen, J. Olle, L. Colmenarez, M. Puviani, M. Müller, and F. Marquardt, “Quantum Circuit Discovery for Fault-Tolerant Logical State Preparation with Reinforcement Learning,” *Physical Review X*, vol. 15, p. 41012, 2025, doi: [10.1103/gqpr-dgz7](https://doi.org/10.1103/gqpr-dgz7).
- [19] Y. Yin, T. Xiao, X. Deng, M. He, J. Fan, and G. Zeng, “Discovering autonomous quantum error correction via deep reinforcement learning,” *Physical Review A*, vol. 112, p. 62618, 2025, doi: [10.1103/rgy3-z928](https://doi.org/10.1103/rgy3-z928).
- [20] M. Webster and D. Browne, “Engineering Quantum Error Correction Codes Using Evolutionary Algorithms,” *IEEE Transactions on Quantum Engineering*, vol. 6, pp. 1–14, 2025, doi: [10.1109/TQE.2025.3538934](https://doi.org/10.1109/TQE.2025.3538934).
- [21] D. Tandeitnik and T. Guerreiro, “Evolving Quantum Circuits,” *Quantum Information Processing*, vol. 23, p. 109, 2024, doi: [10.1007/s11128-024-04317-w](https://doi.org/10.1007/s11128-024-04317-w).
- [22] N. Meyer, C. Mutschler, A. Maier, and D. D. Scherer, “Learning Encodings by Maximizing State Distinguishability: Variational Quantum Error Correction.” [Online]. Available: <https://arxiv.org/abs/2506.11552>

- [23] Y. Shao *et al.*, “Variational Graphical Quantum Error Correction Codes,” *Communications Physics*, 2026, doi: [10.1038/s42005-026-02681-w](https://doi.org/10.1038/s42005-026-02681-w).
- [24] N. Meyer, C. Mutschler, D. Seuß, A. Maier, and D. D. Scherer, “Learning to Concatenate Quantum Codes.” [Online]. Available: <https://arxiv.org/abs/2604.14931>
- [25] M. Casanova, K. Ohki, and F. Ticozzi, “Finding Quantum Codes via Riemannian Optimization,” *Quantum Science and Technology*, vol. 10, p. 25027, 2025, doi: [10.1088/2058-9565/adb3c6](https://doi.org/10.1088/2058-9565/adb3c6).
- [26] M. Seksaria and A. Prabhakar, “Correcting quantum errors one gradient step at a time.” [Online]. Available: <https://arxiv.org/abs/2512.18061>
- [27] Y. Chengyu, R. Meister, C. Carty, S.-K. Lin, and R. Bondesan, “Bayesian Optimization for Quantum Error-Correcting Code Discovery.” [Online]. Available: <https://arxiv.org/abs/2601.18562>
- [28] M. Wang and F. Mueller, “Coprime Bivariate Bicycle Codes and Their Layouts on Cold Atoms,” *Quantum*, vol. 10, p. 2009, 2026, doi: [10.22331/q-2026-02-23-2009](https://doi.org/10.22331/q-2026-02-23-2009).
- [29] J. Cruz-Benito, A. W. Cross, D. Kremer, and I. Faro, “Evolutionary Discovery of Bivariate Bicycle Codes with LLM-Guided Search.” [Online]. Available: <https://arxiv.org/abs/2606.02418>
- [30] B. C. B. Symons, A. Rajput, and D. E. Browne, “Sequences of Bivariate Bicycle Codes from Covering Graphs.” [Online]. Available: <https://arxiv.org/abs/2511.13560>
- [31] L. Voss, S. J. Xian, T. Haug, and K. Bharti, “Multivariate Bicycle Codes,” *Physical Review A*, vol. 111, p. L60401, 2025, doi: [10.1103/PhysRevA.111.L60401](https://doi.org/10.1103/PhysRevA.111.L60401).
- [32] A. A. Galimova, “Independent Trivariate Bicycle Codes.” [Online]. Available: <https://arxiv.org/abs/2603.17703>
- [33] A. Jacob, C. McLauchlan, and D. E. Browne, “Single-Shot Decoding and Fault-tolerant Gates with Trivariate Tricycle Codes.” [Online]. Available: <https://arxiv.org/abs/2508.08191>
- [34] F. A. Mian, O. Gwilliam, and S. Krastanov, “Multivariate Multicycle Codes for Complete Single-Shot Decoding.” [Online]. Available: <https://arxiv.org/abs/2601.18879>
- [35] R. Wang, H.-K. Lin, and L. P. Pryadko, “Abelian and non-abelian quantum two-block codes,” *2023 12th International Symposium on Topics in Coding (ISTC)*, pp. 1–5, 2023, doi: [10.1109/ISTC57237.2023.10273492](https://doi.org/10.1109/ISTC57237.2023.10273492).
- [36] S. P. Jain and V. V. Albert, “Transversal Clifford and T-Gate Codes of Short Length and High Distance,” *IEEE Journal on Selected Areas in Information Theory*, vol. 6, pp. 127–137, 2025, doi: [10.1109/JSait.2025.3570832](https://doi.org/10.1109/JSait.2025.3570832).
- [37] J. J. Postema and S. J. J. M. F. Kokkermans, “Existence and Characterisation of Bivariate Bicycle Codes.” [Online]. Available: <https://arxiv.org/abs/2502.17052>
- [38] V. Guémard and G. Zémor, “Moderate-Length Lifted Quantum Tanner Codes.” [Online]. Available: <https://arxiv.org/abs/2502.20297>
- [39] N. Raveendran, D. Declercq, and B. Vasić, “On the Minimum Distances of Finite-Length Lifted Product Quantum LDPC Codes.” [Online]. Available: <https://arxiv.org/abs/2503.07567>
- [40] S. Ashhab, “Automated discovery and optimization of autonomous quantum error correction codes for a general open quantum system,” *New Journal of Physics*, vol. 28, p. 24503, 2026, doi: [10.1088/1367-2630/ae4135](https://doi.org/10.1088/1367-2630/ae4135).
- [41] L. P. Pryadko, V. A. Shabashov, and V. K. Kozin, “QDistRnd: A GAP package for computing the distance of quantum error-correcting codes,” *Journal of Open Source Software*, vol. 7, p. 4120, 2022, doi: [10.21105/joss.04120](https://doi.org/10.21105/joss.04120).
- [42] Z. Liang, Z. Wang, Z. Yi, Y. Wu, C. Qiu, and X. Wang, “Determining the upper bound of code distance of quantum stabilizer codes through Monte Carlo method based on fully decoupled belief propagation.” [Online]. Available: <https://arxiv.org/abs/2402.06481>
- [43] R. Ismail, A. Kakkar, and A. Dymarsky, “A quantum annealing approach to the minimum distance problem of quantum codes.” [Online]. Available: <https://arxiv.org/abs/2404.17703>
- [44] J. Haah, “Classification of Translation Invariant Topological Pauli Stabilizer Codes for Prime Dimensional Qudits on Two-Dimensional Lattices,” *Journal of Mathematical Physics*, vol. 62, no. 1, 2021, doi: [10.1063/5.0021068](https://doi.org/10.1063/5.0021068).
- [45] M. A. Perlin and qLDPC contributors, “qLDPC: Tools for Quantum Low-Density Parity-Check Codes.” [Online]. Available: <https://github.com/qLDPCOrg/qLDPC>
- [46] A. Cowtan, “SSIP: automated surgery with quantum LDPC codes.” [Online]. Available: <https://arxiv.org/abs/2407.09423>

- [47] N. Aydin, P. Liu, and B. Yoshino, “A Database of Quantum Codes,” *Journal of Algebra Combinatorics Discrete Structures and Applications*, pp. 185–191, 2022, doi: [10.13069/jacodesmath.v9i3.217](https://doi.org/10.13069/jacodesmath.v9i3.217).
- [48] V. V. Albert and P. Faist, “The Error Correction Zoo.” [Online]. Available: <https://errorcorrectionzoo.org/>
- [49] Z. Wang and H. Tang, “Artificial Intelligence for Quantum Error Correction: A Comprehensive Review.” [Online]. Available: <https://arxiv.org/abs/2412.20380>
- [50] Q. Huang, L. Zhou, W. Fang, M. Zhao, and M. Ying, “Efficient Formal Verification of Quantum Error Correcting Programs,” *Proceedings of the ACM on Programming Languages*, vol. 9, pp. 1068–1093, 2025, doi: [10.1145/3729293](https://doi.org/10.1145/3729293).